ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

ARQUITECTURAS DE SOFTWARE · 2026-2

Laboratorio de observabilidad y pruebas de rendimiento

Escuela Colombiana de Ingeniería Julio Garavito

105 minutos · Trabajo en parejas · Entrega al finalizar la sesión

Los puntos del taller están integrados en este laboratorio. No hay una entrega adicional.

Propósito

El equipo administra una API de consulta de tableros. Debe demostrar cuándo está disponible, qué rendimiento ofrece y cómo se comporta bajo diferentes cargas. Construya un tablero en Grafana y tome decisiones con evidencia de k6. No necesita programar la aplicación ni instalar complementos de Grafana.

Resultados de aprendizaje

- Construir un tablero que distinga disponibilidad funcional, rendimiento y uso de recursos.
- Ejecutar pruebas de carga, estrés, pico y duración, y explicar sus resultados.
- Comparar una mejora controlada con una línea base y sustentar conclusiones con evidencia.

Ruta de trabajo

Minutos	Actividad	Evidencia
0–10	Arranque y comprobación	Servicios disponibles
10–35	Construcción del tablero	Ocho paneles con unidades
35–45	Falla y recuperación	Captura y explicación
45–70	Cuatro pruebas automatizadas	Resultados JSON y tabla
70–85	Experimento de mejora	Comparación antes y después
85–100	Análisis integrado	Respuestas y conclusión
100–105	Exportación y entrega	ZIP completo

Objetivos para la carga normal

A 20 solicitudes/s: P95 < 500 ms, errores < 1 % y verificaciones correctas > 99 %. Son criterios didácticos del ejercicio, no acuerdos contractuales de producción.

02 / LABORATORIO ARSW

Preparar el entorno

Antes de clase · La descarga inicial queda fuera de los 105 minutos

Qué necesita cada pareja

Docker Engine o Docker Desktop en ejecución con contenedores Linux y Compose v2; Python 3.10 o superior; navegador e Internet para la primera descarga. Se sugieren 4 GB de RAM disponibles para Docker y 3 GB de disco. No se requieren paquetes pip, cuenta de Grafana Cloud ni k6 instalado en el equipo.

1 Abrir el proyecto y activar Python

Descomprima el kit y abra una terminal en estudiantes. En macOS con zsh, cree el entorno una sola vez y actívelo en cada terminal nueva:

```
python3 -m venv .venv
source .venv/bin/activate
python --version
```

En Windows con PowerShell: cree el entorno con py -m venv .venv y actívelo con .\.venv\Scripts\Activate.ps1. Si ya tiene un entorno activo, úselo.

2 Descargar y levantar los servicios

```
python scripts/lab.py prepare
python scripts/lab.py up
```

Espere unos 15 segundos y compruebe el arranque:

```
python scripts/lab.py doctor
```

Si los servicios siguen iniciando, espere y repita doctor. Mantenga libres los puertos 3000, 8000 y 9090. Las pruebas se realizan únicamente sobre este entorno local.

3 Abrir las herramientas

Servicio	Dirección o acceso
Grafana	http://localhost:3000 Usuario: admin · Clave: arsw-lab-2026
API	http://localhost:8000/api/boards/1
Prometheus	http://localhost:9090

Cómo funciona el escenario

La API Python consulta SQLite y emula una dependencia con una espera de 120 ms. Tiene ocho cupos simultáneos; cuando se agotan, rechaza solicitudes con HTTP 503. La espera artificial no corresponde al tiempo de consulta de SQLite.

Prometheus recoge métricas cada 5 s. Una sonda independiente consulta /ready aproximadamente cada 2 s. Grafana consulta Prometheus y k6 genera tráfico hacia la API. La fuente de datos está configurada; los estudiantes construyen su tablero.

03 / LABORATORIO ARSW

Construir el tablero y observar una falla

Parte 1 · Minutos 10 a 45

1 Generar tráfico y crear el tablero

```
python scripts/lab.py sample
```

En Grafana: Dashboards → New dashboard → Add visualization → Prometheus. Seleccione Code y use las expresiones de CONSULTAS.md. Construya estos ocho paneles:

Panel	Visualización y unidad
Lectura de métricas del servicio	Stat · up: 0 o 1
Disponibilidad funcional actual	Stat · sonda: 0 o 1
Disponibilidad observada en 5 min	Stat · porcentaje 0–100
Errores HTTP	Time series · porcentaje; referencia 1 %
Throughput total	Time series · solicitudes/s
P95 del servidor	Time series · segundos; referencia 0.5 s
Ocupación de capacidad	Time series · porcentaje 0–100
CPU del proceso	Time series · porcentaje de un núcleo

Guarde como ARSW Observabilidad Equipo XX. Use últimos 15 minutos y actualización cada 5 s. Configure 0 en rojo y 1 en verde para los estados.

2 Provocar la falla y observar la recuperación

```
python scripts/lab.py fault
```

Durante 30 s, la API y /ready fallan, pero /metrics responde. Observe up y lab_probe_success; capture el intervalo de falla y recuperación. Después, genere tráfico de nuevo con sample.

3 Registrar la evidencia

Anote la disponibilidad observada, el intervalo de falla y el tiempo aproximado hasta que la sonda vuelve a 1. Explique por qué up = 1 no demuestra disponibilidad funcional. Este tiempo de recuperación no es un MTTR histórico.

Claves para interpretar los paneles

La disponibilidad corresponde a sondeos exitosos en una ventana de 5 min; no es un SLA mensual. rate e increase requieren varias muestras: espere al menos un minuto para interpretar tendencias. La resolución de 5 s puede ocultar picos breves de ocupación.

No convierta ausencia de datos en cero: sin tráfico no hay P95 ni porcentaje de errores significativo. El throughput total incluye respuestas fallidas.

04 / LABORATORIO ARSW

Ejecutar las cuatro pruebas

Parte 2 · Minutos 45 a 70

1 Revisar el escenario y ejecutar la suite

Revise k6/tests.js. Cada iteración envía una solicitud y verifica HTTP 200 y el id del tablero. Los perfiles programan una tasa de llegada; los VUs son trabajadores disponibles para sostenerla, no la tasa misma.

```
python scripts/lab.py reset
python scripts/lab.py test all
```

Perfil	Tráfico ofrecido	Duración	Pregunta experimental
carga	20 solicitudes/s	1 min	¿Cumple el objetivo normal?
estres	20 → 60 → 120 → 20 solicitudes/s	2 min	¿Cuándo se degrada y se recupera?
pico	20 → 160 → 20 solicitudes/s	90 s	¿Tolera un aumento abrupto?
duracion	20 solicitudes/s	4 min	¿Hay deriva visible a carga constante?

La suite tarda aproximadamente nueve minutos, incluidos cierres. La prueba de duración es una versión corta de resistencia o soak: 4 min no demuestran estabilidad durante horas ni ausencia de fugas.

2 Leer los resultados de cada ejecución

results/ contiene un JSON de métricas y un archivo compañero -run.json con inicio y fin UTC. Conserve los originales. Complete ENTREGA.md para los cuatro perfiles con estos campos dentro de metrics:

Dato	Campo del JSON	Unidad
P95	http_req_duration.values["p(95)"]	ms
Errores	http_req_failed.values.rate	Fracción
Throughput	http_reqs.values.rate	Solicitudes/s
Descartadas	dropped_iterations.values.count	Iteraciones

Si dropped_iterations no aparece, registre 0. Si es mayor que cero, el generador no inició toda la carga programada: no atribuya automáticamente el resultado a la API. Multiplique las fracciones por 100 para informar porcentajes.

3 Interpretar el resultado

El código 99 indica umbrales incumplidos; puede ser esperable en estrés o pico. Otro código requiere revisar la ejecución. Se evalúan el experimento y la interpretación, no que todo quede verde.

Compare las ventanas de Grafana con las horas UTC registradas. El P95 de k6 mide desde el cliente; el del servidor estima tiempo interno mediante intervalos y excluye parte del transporte. No deben ser idénticos. El P95 global no es el promedio de los P95 por minuto.

05 / LABORATORIO ARSW

Comparar una mejora y explicar los hallazgos

Minutos 70 a 100 · Experimento y preguntas integradas

1 Formular la predicción

Use la primera prueba de estrés como línea base. Prediga qué ocurrirá al reducir la espera simulada de la dependencia de 120 a 60 ms, manteniendo ocho cupos y el mismo perfil. Registre la predicción antes de ejecutar.

2 Cambiar una variable y repetir

```
python scripts/lab.py improve
python scripts/lab.py test estres
python scripts/lab.py reset
```

Compare errores, throughput, P95 y recuperación. Mantenga equipo, perfil, versión y configuración restante iguales. Conserve ambas ejecuciones y registre qué cambió.

3 Contrastar la predicción con los datos

Como referencia teórica, capacidad ≈ cupos / tiempo de servicio: 8/0.12 ≈ 67 solicitudes/s y 8/0.06 ≈ 133 solicitudes/s. No son resultados garantizados. La intervención emula una mejora de la dependencia; no demuestra una optimización real de SQLite ni escalamiento horizontal.

Preguntas integradas del taller

Responda en ENTREGA.md en dos o tres frases por pregunta. Use cifras, unidades, ventanas y capturas de su propia ejecución.

1. ¿Qué significa P95 y por qué un promedio bajo puede esconder solicitudes lentas? Interprete su P95 de carga con unidad y ventana.

2. Durante la falla, ¿por qué up puede valer 1 y el sondeo funcional 0? ¿Cuál usaría para informar al usuario sobre disponibilidad?

3. ¿Cuál es la diferencia entre carga, estrés, pico y duración? Vincule cada prueba con una observación concreta.

4. Si CPU está baja y aumentan rechazos, ¿qué otra limitación puede existir en esta API? Relacione espera, cupos, tasa y VUs.

5. ¿La mejora cumplió su predicción? ¿Qué experimento adicional necesitaría para afirmar escalabilidad o estabilidad prolongada?

Antes de concluir

Lea latencia y errores juntos: un rechazo rápido puede reducir el P95 agregado sin mejorar el servicio. Distinga carga ofrecida de carga ejecutada y reconozca las limitaciones del equipo, el muestreo y la corta duración de las pruebas.

06 / LABORATORIO ARSW

Entregar la evidencia y verificar la rúbrica

Minutos 100 a 105 · Entrega en clase

Un único archivo por pareja

Nombre de entrega: ARSW_Lab_EquipoXX.zip

Contenido	Verificación antes de enviar
ENTREGA.md	Integrantes, características del equipo, cinco ejecuciones, comparación y cinco respuestas.
dashboard.json	Tablero construido por la pareja, exportado e importable.
results/	JSON de métricas y archivos -run.json de carga, estrés base, pico, duración y estrés con mejora.
capturas/	Tablero normal, falla/recuperación y comparación estrés antes/después. Esta última puede unir dos capturas.

Exportar y comprobar el tablero

En el tablero guardado, abra Settings/JSON Model o Export, según la interfaz. Guarde dashboard.json. Compruebe la importación en Dashboards → New → Import y que conserve Prometheus como fuente. Una captura no sustituye el JSON; no incluya volúmenes de Docker.

Rúbrica sobre 5 puntos

Criterio	Puntos	Evidencia de logro
Arranque reproducible	0.50	Configuración, equipo e identificación
Tablero de observabilidad	1.50	Ocho paneles correctos, unidades y rangos
Disponibilidad y recuperación	0.75	Falla observada y distinción up/sondeo
Cuatro pruebas automatizadas	1.00	Perfiles, JSON y tabla completos
Mejora y análisis integrado	1.00	Comparación controlada y cinco respuestas
Entrega reproducible	0.25	JSON importable y archivos organizados

Se otorga crédito parcial por evidencia correcta aunque un umbral falle. Un panel sin unidad o con consulta incorrecta no obtiene el puntaje completo. Las conclusiones deben reconocer limitaciones y no inventar mediciones.

Cerrar el entorno después de entregar

```
python scripts/lab.py down
```

Exporte antes de borrar volúmenes. Este comando conserva el volumen de Grafana; el histórico de Prometheus no se conserva al recrear su contenedor. Mantenga los JSON de results/ como evidencia.